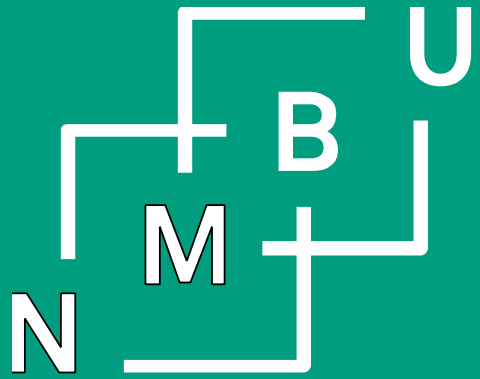



Norwegian University
of Life Sciences

# Cross-validation and parameter optimisation

Chapter 6 in the Python Machine Learning book

# Overview

- Recap:
  - Hyperparameters
  - The holdout method (“validation” partition)
- *k*-fold cross-validation
- Alternative cross-validation strategies
- GridSearchCV and RandomizedSearchCV
- Interpretation of learning and validation curves. How to detect:
  - Underfitting (high bias)
  - Overfitting (high variance)

# Resources

- Python Machine Learning TE, Chapter 6, pages 191 – 222
  - Jupyter notebook: Chapter 6, part 1

- Model selection, tuning and evaluation in scikit-learn: http://scikit-learn.org/stable/model_selection.html

# Hyperparameters

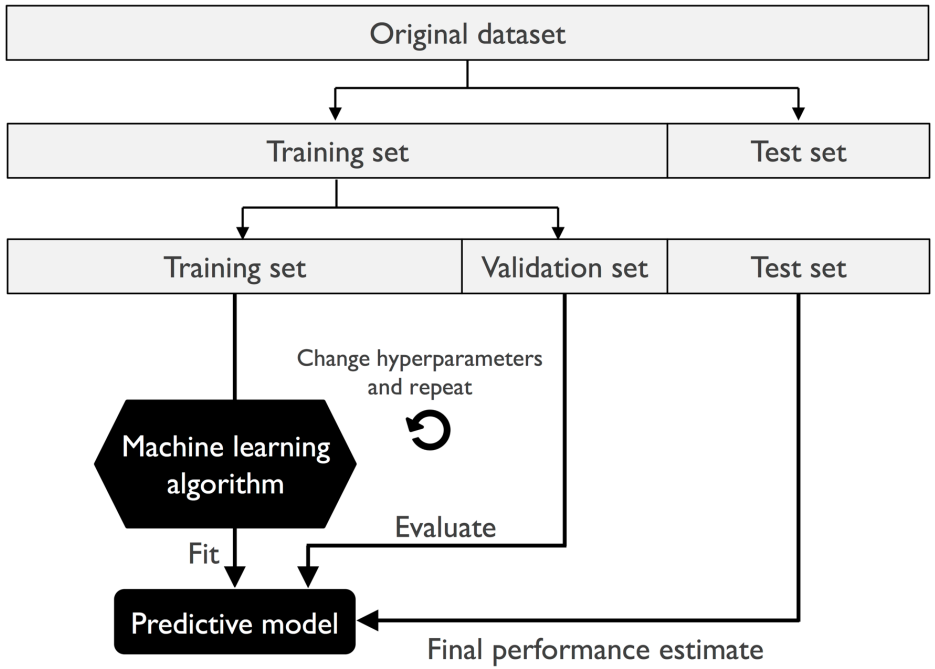
- Hyperparameters are **non-trainable** model parameters that have an affect on a model´s performance
  - Examples include max depth in a decision tree, the number of decision trees in a random forest model, any regularization technique, etc.
  - No clear distinction between what is considered a “hyperparameter”, and what is considered a different model architecture entirely
- Hyperparameter optimization is also referred to as *model tuning*

## The holdout method (“validation” partition)

- Earlier we talked about the need for a third partition when splitting the dataset
- Does anybody remember **why** we wanted the additional “validation” partition?
  - If we use the test-partition to adjust a model´s hyperparameters, we are indirectly training the model on the test-partition
  - Thus the test-partition is **no longer “unseen”** by the model and we risk **overfitting** the model **on the test set** if we also use the test-partition for final evaluation of the model
- Despite this, many still use the test-partition for both model tuning and final model evaluation. This is **bad practice!**

# The holdout method (“validation” partition)

The method of splitting the full dataset into a training partition and an evaluation partition is referred to as the *holdout method*




Stop here for holdout validation

When splitting the training set again it’s called **holdout cross-validation**

# The holdout method (“validation” partition)

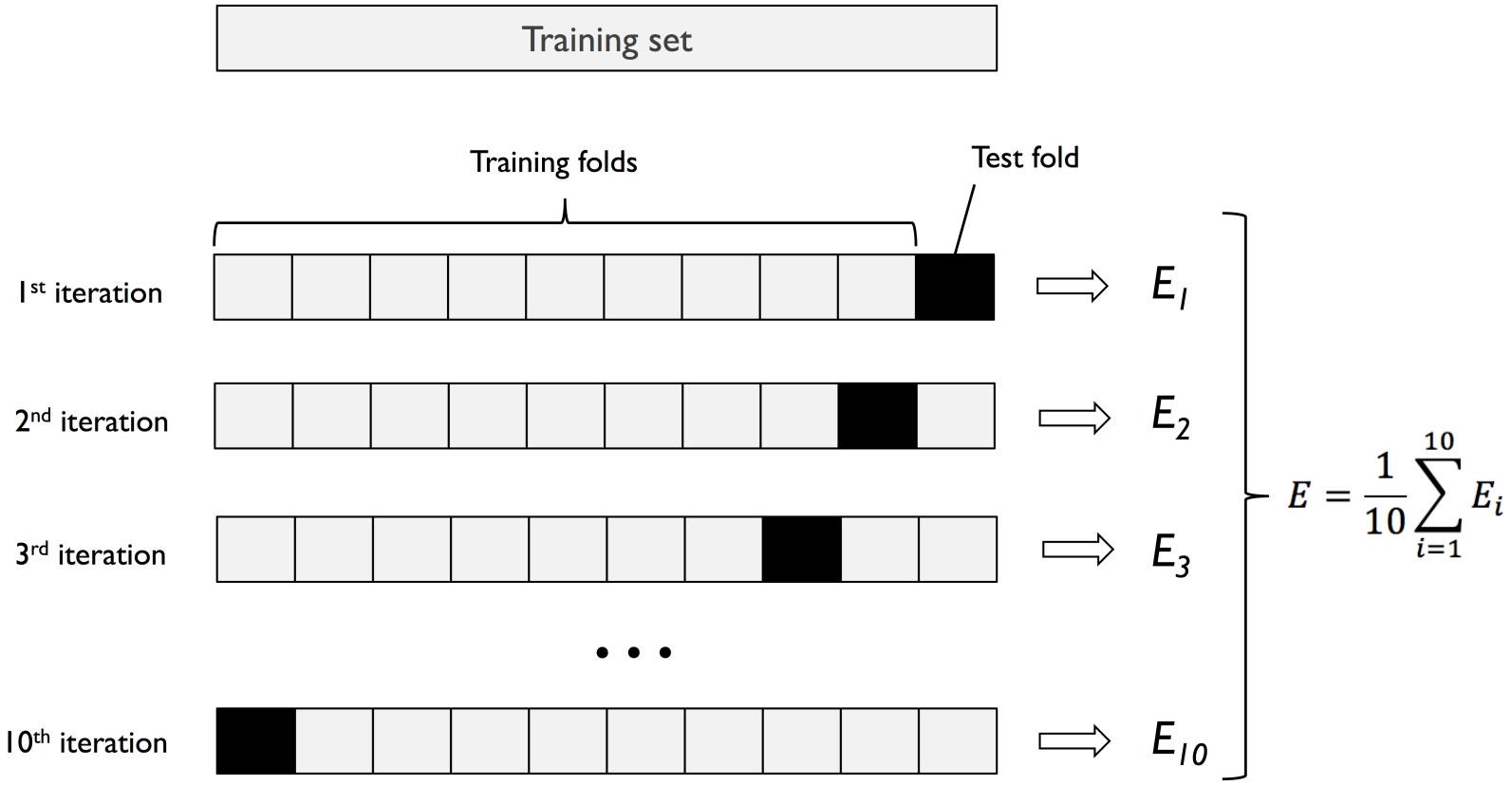
- The big disadvantage with holdout cross-validation is that the **performance** of the particular model hyperparameter configurations could be **highly sensitive** to **how** the **training set is partitioned** into training and validation sets.

- This leads us to the second variation of cross-validation:
  ***k*-fold cross-validation**

# *k*-fold cross-validation (CV)

- For *k*-fold CV we still perform the initial holdout split into training set and test set
- We then partition the training set into *k* folds
- *k*-fold cross-validation is then performed in *k* iterations. For each iteration (*i*):
  - Set the validation partition equal to fold *i*
  - Reset the model´s trainable parameters
  - Train the model on the remaining *k – 1* folds
  - Evaluate the model on the validation partition and store the accuracy (or other metric)
- After all iterations are complete → Compute the average accuracy
- An additional plus is that we can compute the **uncertainty** of the model performance

# *k*-fold cross-validation (CV)



## *k*-fold cross-validation (CV)

- Higher values of *k*:
  - Model validation becomes **more thorough**
  - Model validation becomes more **computationally expensive**
- Empirical evidence shows that *k*=10 is a good starting point
  - A compromise between bias and model variance, and computational cost
- If working with a relatively **small dataset: *k* > 10**
- If working with a relatively **large dataset: *k* < 10**

## *K*-fold cross-validation (CV)

- Important to maintain the distributions of the dataset in the folds
  - *StratifiedKFold* from *sklearn.model_selection*
  - *cross_val_score* from *sklearn.model_selection*
- Leave-one-out-cross-validation (LOOCV)
  - Special case of *k*-fold CV where *k* = *n* (the total number of samples)
- *cross_val_score* has a parameter *n_jobs* which specifies the number of CPU cores used for CV -> Can be used to speed up the computations

Jupyter: Chapter 6, part 1

# Cross-validation

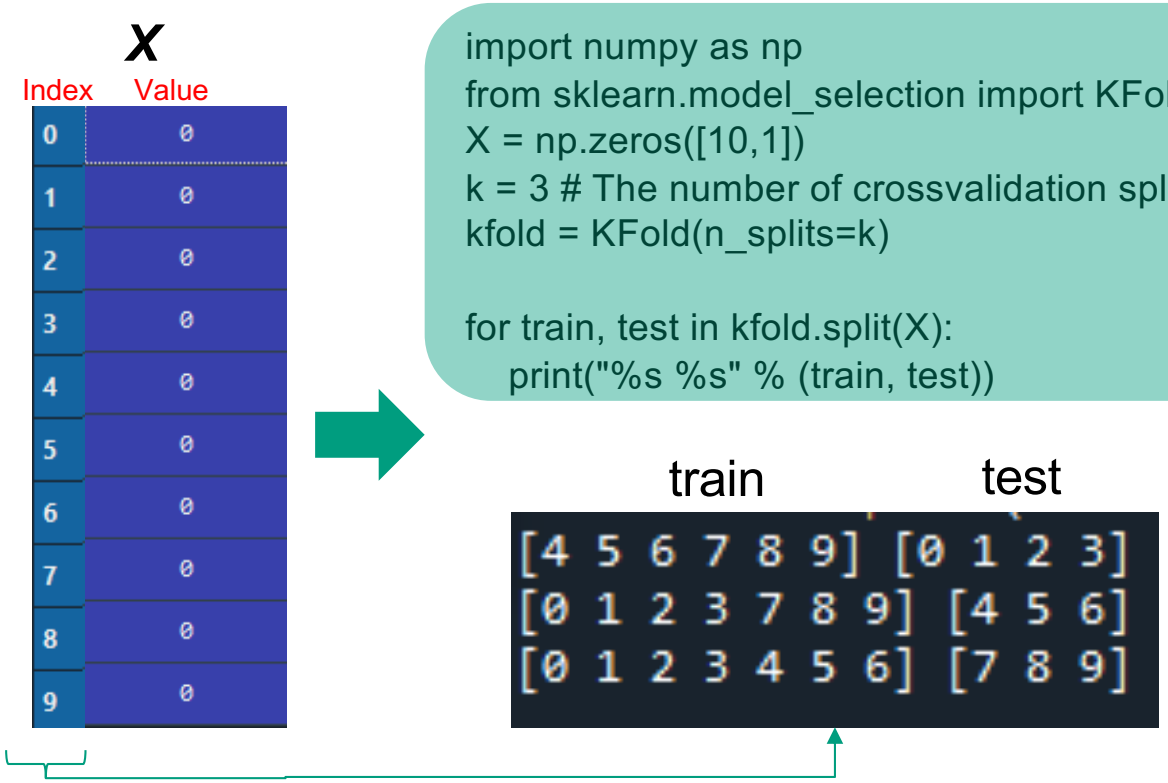
- Search for optimum parameters
- Assess stability

- Strategies:
  - *K*-fold (consecutive)
  - ShuffleSplit
  - LOO (leave-one-out)
  - LeavePGroupsOut
  - Stratified versions (folds within categories or other strata)
  - …
- Number of folds?

# Cross-validation

Strategies:

– *K*-fold




Code Example

```
import numpy as np
from sklearn.model_selection import KFold
X = np.zeros([10,1])
k = 3 # The number of crossvalidation splits
kfold = KFold(n_splits=k)

for train, test in kfold.split(X):
    print("%s %s" % (train, test))
```

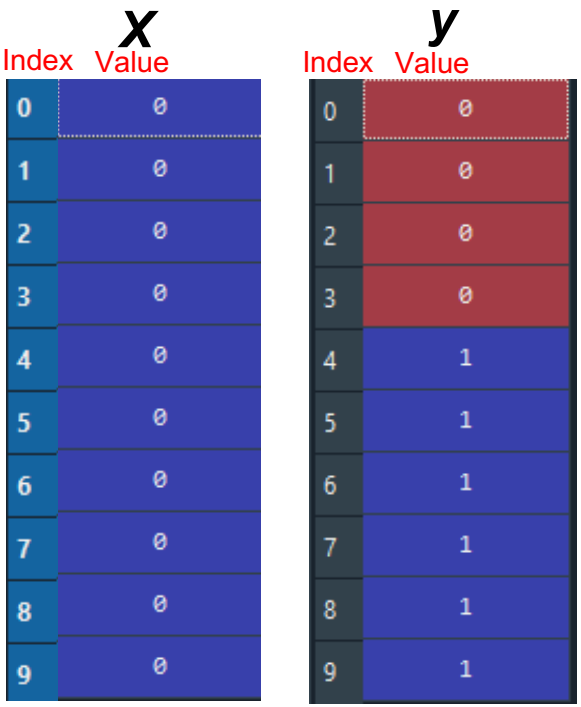
train test

```
[4 5 6 7 8 9] [0 1 2 3]
[0 1 2 3 7 8 9] [4 5 6]
[0 1 2 3 4 5 6] [7 8 9]
```

# Cross-validation

Strategies:

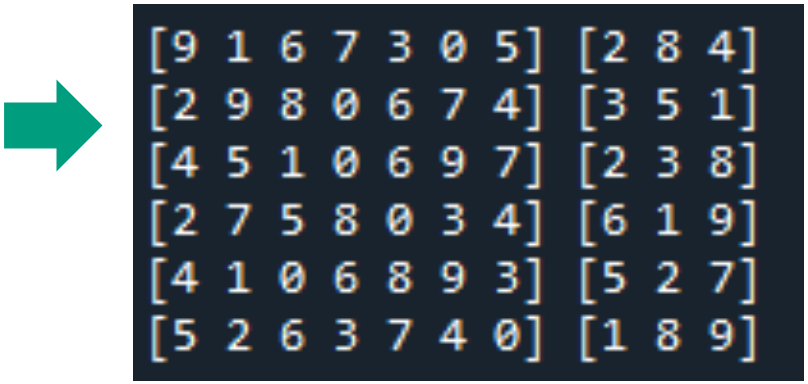
- Shuffle Split




## Code Example

```
from sklearn.model_selection import
ShuffleSplit

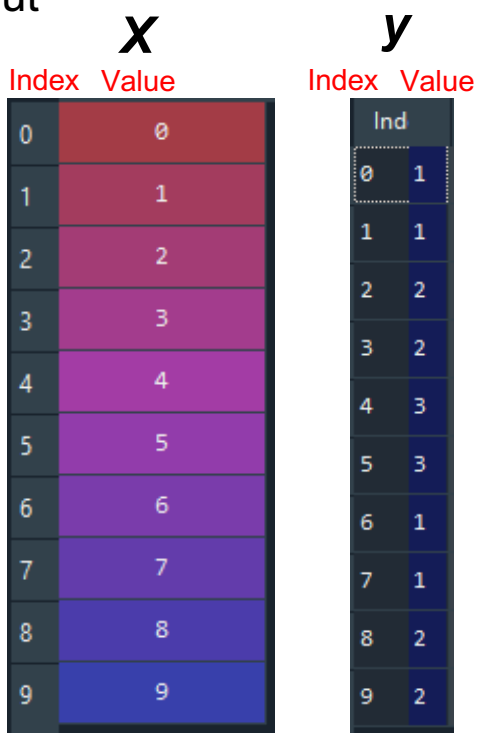
# Two classes to be represented in each split
X = np.zeros([10,1])
y = np.hstack([np.zeros(4),np.ones(6)])
ss = ShuffleSplit(n_splits=6,
                  test_size=0.25, random_state=0)
print("%s %s" % (train, test))
```



# Cross-validation

Strategies:

- LeaveOneOut




## Code Example

```
from sklearn.model_selection import
LeaveOneOut
import numpy as np

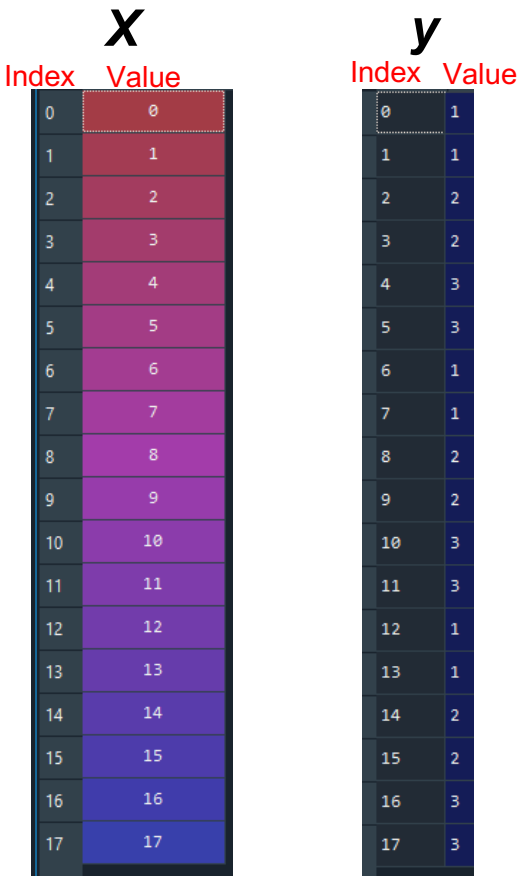
loo = LeaveOneOut()
X = np.arange(10)
y = [1, 1, 2, 2, 3, 3, 1, 1, 2, 2]
for train, test in loo.split(X, y):
    print("%s %s" % (train, test))
```

```
[1 2 3 4 5 6 7 8 9] [0]
[0 2 3 4 5 6 7 8 9] [1]
[0 1 3 4 5 6 7 8 9] [2]
[0 1 2 4 5 6 7 8 9] [3]
[0 1 2 3 5 6 7 8 9] [4]
[0 1 2 3 4 6 7 8 9] [5]
[0 1 2 3 4 5 7 8 9] [6]
[0 1 2 3 4 5 6 8 9] [7]
[0 1 2 3 4 5 6 7 9] [8]
[0 1 2 3 4 5 6 7 8] [9]
```

# Cross-validation

Strategies:

- LeavePGroupsOut




## Code Example

```
from sklearn.model_selection import
LeavePGroupsOut
import numpy as np

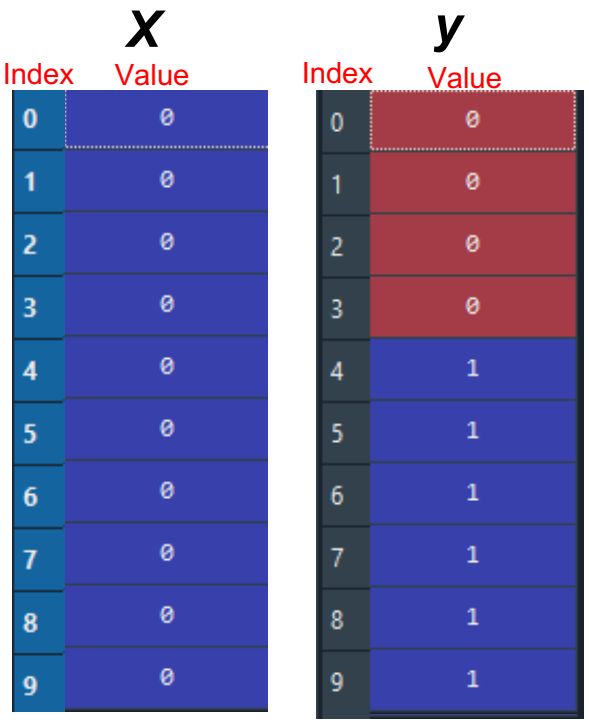
lpgo = LeavePGroupsOut(n_groups=2)
X = np.arange(18)
y = [1, 1, 2, 2, 3, 3, 1, 1, 2, 2, 3, 3, 1, 1, 2,
2, 3, 3]
segments = [1, 1, 2, 2, 3, 3, 4, 4, 5, 5, 6, 6,
7, 7, 8, 8, 9, 9]
for train, test in lpgo.split(X, y,
groups=segments):
    print("%s %s" % (train, test))
```

```
[ 4  5  6  7  8  9 10 11 12 13 14 15 16 17] [0 1 2 3]
[ 2  3  6  7  8  9 10 11 12 13 14 15 16 17] [0 1 4 5]
[ 2  3  4  5  8  9 10 11 12 13 14 15 16 17] [0 1 6 7]
[ 2  3  4  5  6  7 10 11 12 13 14 15 16 17] [0 1 8 9]
[ 2  3  4  5  6  7  8  9 12 13 14 15 16 17] [ 0  1 10 11]
[ 2  3  4  5  6  7  8  9 10 11 14 15 16 17] [ 0  1 12 13]
[ 2  3  4  5  6  7  8  9 10 11 12 13 16 17] [ 0  1 14 15]
[ 2  3  4  5  6  7  8  9 10 11 12 13 14 15] [ 0  1 16 17]
[ 0  1  6  7  8  9 10 11 12 13 14 15 16 17] [2 3 4 5]
[ 0  1  4  5  8  9 10 11 12 13 14 15 16 17] [2 3 6 7]
[ 0  1  4  5  6  7 10 11 12 13 14 15 16 17] [2 3 8 9]
```

# Cross-validation

Strategies:

– Stratified




Code Example

```
from sklearn.model_selection import StratifiedKFold

# Two classes to be represented in each split
y = np.hstack([np.zeros(4),np.ones(6)])
kfold = StratifiedKFold(n_splits=3).split(X, y)
for k, (train, test) in enumerate(kfold):
    print("%s %s" % (train, test))
```

train test

```
[2 3 6 7 8 9] [0 1 4 5]
[0 1 3 4 5 8 9] [2 6 7]
[0 1 2 4 5 6 7] [3 8 9]
```

# Cross-validation techniques - summary

| Technique | How It Works | When To Use |
|---|---|---|
| K-Fold Cross-Validation | Splits data into K equal parts (folds), each fold is tested once. | Balanced datasets. Small to medium-sized datasets. |
| Stratified K-Fold Cross-Validation | Same as K-Fold, but preserves class distribution across folds. | Classification with imbalanced data. |
| Shuffle Split | Randomly shuffles data before splitting. Creates multiple, independent splits. | Small datasets. When repeated random sampling is needed. |
| Leave-One-Out Cross-Validation (LOOCV) | Each data point is tested once, with the rest used for training. | Very small datasets. |
| Leave-P-Groups-Out Cross-Validation (LPGOCV) | Leaves out entire groups for testing while training on the rest. | Data grouped by patients, experiments, or time periods. |

# Learning and validation curves

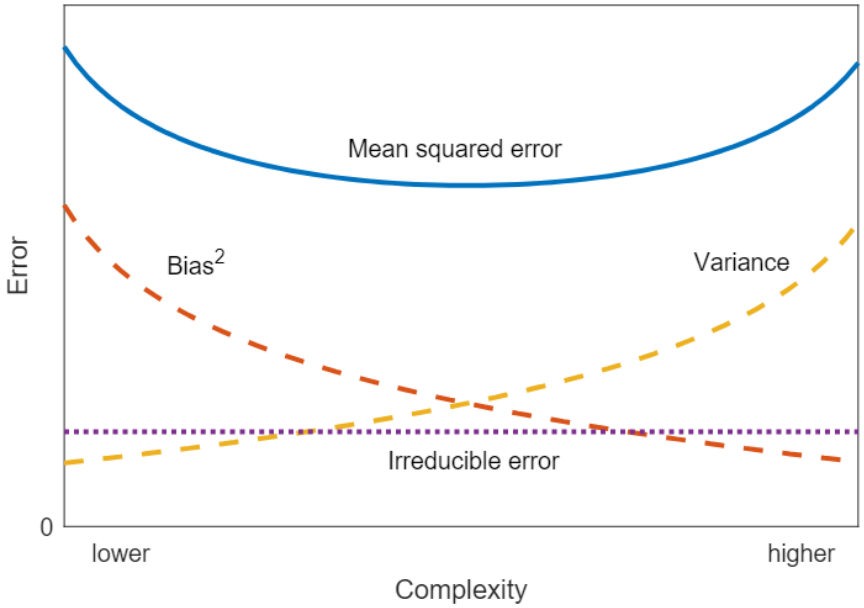
- A learning curve is a plot of a model´s **accuracy vs. # samples** in the training set
  - Training accuracy vs. the number of training samples
  - Validation accuracy vs. the number of training samples
- Validation curves are related to learning curves, but instead of varying the number of training samples, we vary the values of the model parameters

# Recap: Bias/Variance (underfitting/overfitting)

- High variance: The model is too complex → too many degrees of freedom or trainable parameters in the model
  - The model tends to do well (overfit) on the training data
  - Does not generalize well to unseen data
  - **Big gap between the training accuracy and the validation accuracy**
- High bias: The model is not complex enough → too few trainable parameters for this model, or too strict regularization constraints
  - **The model performs poorly on both the training data and unseen data**

# Bias-Variance trade-off

- Bias:
  - Average deviation from the target
- Variance:
  - Spread around the average/target
- Bias <-> Variance are usually highly connected



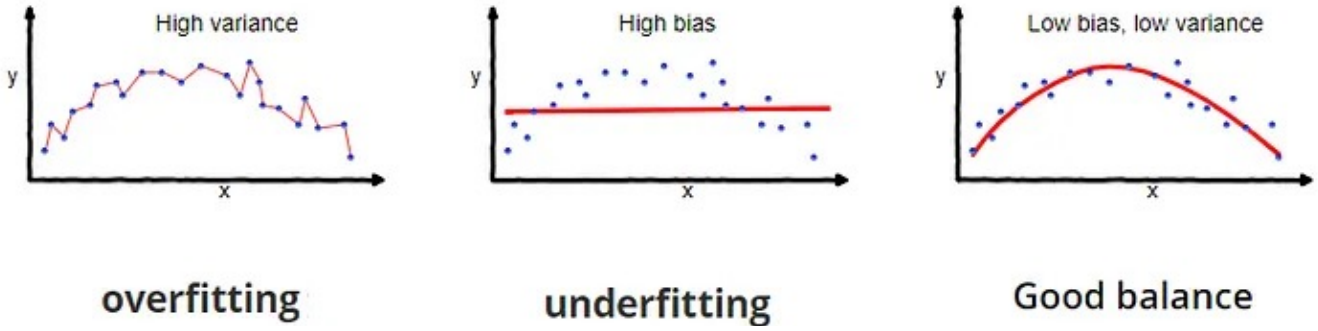
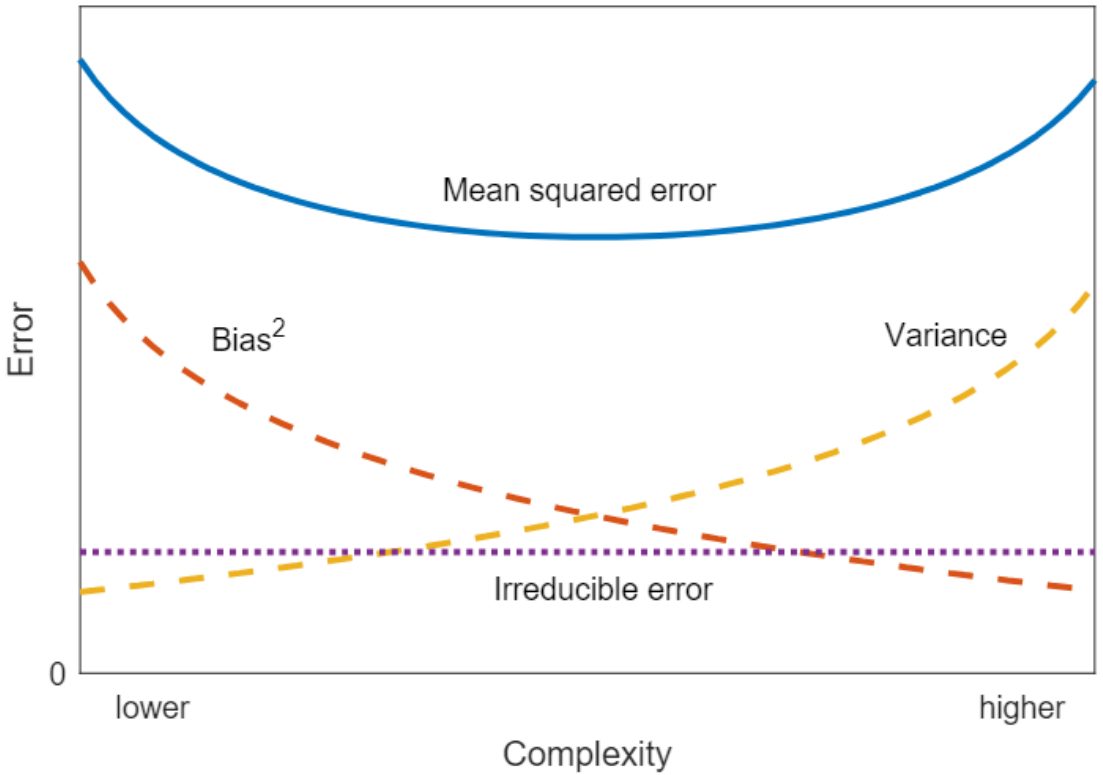



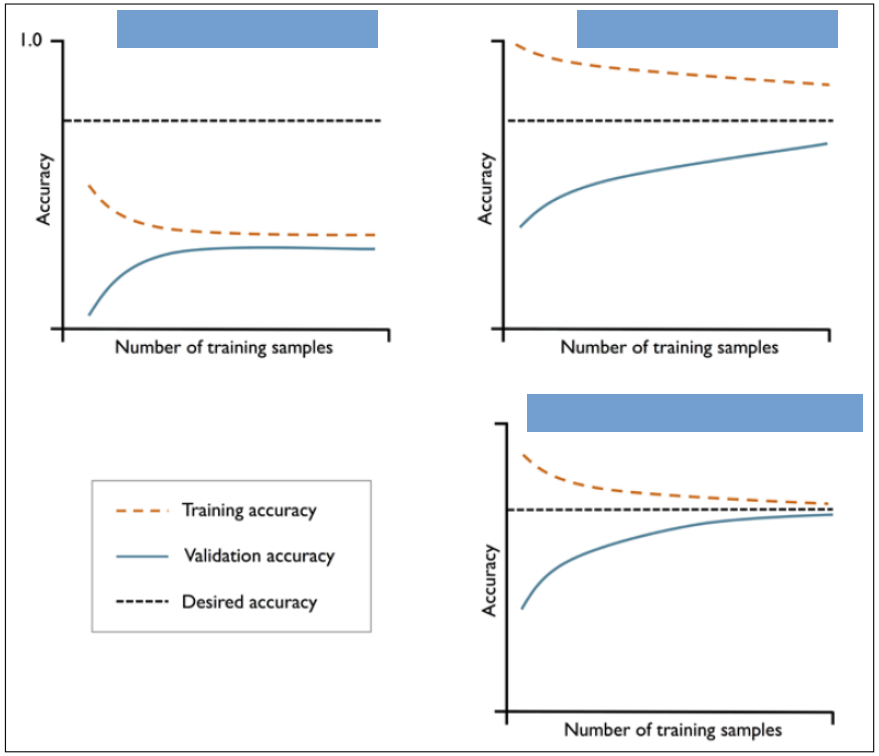

Image: https://towardsdatascience.com/understanding-the-bias-variance-tradeoff-165e6942b229

# Bias-Variance trade-off

- The aim is to find the optimal trade-off between bias and variance that **minimizes the overall error**
- Since there will always be a level of irreducible error, the desired accuracy is set to less than 1 in the following illustrations

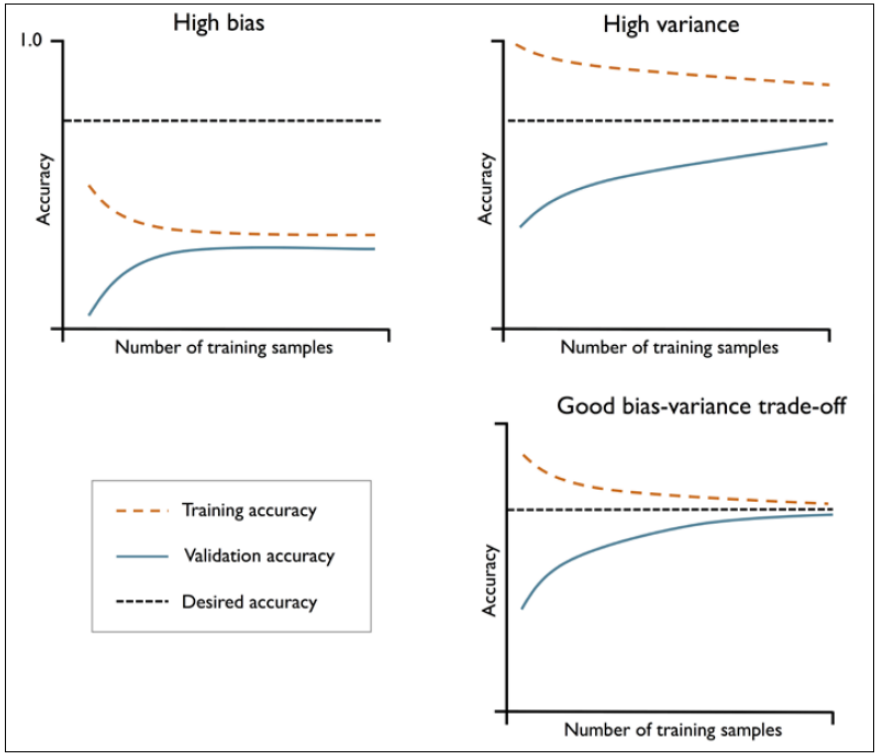


# Learning curves



# Learning curves

Underfitting, try:
- Construct or collect more features
- More flexible modelling
- Less regularisation




Overfitting, try:
- More data / fewer features
- Less flexible modelling
- More regularization

Balanced fitting, try:
- Pat yourself on the back
- Perform victory dance
- Brag about it

Jupyter: Chapter 6, part 1

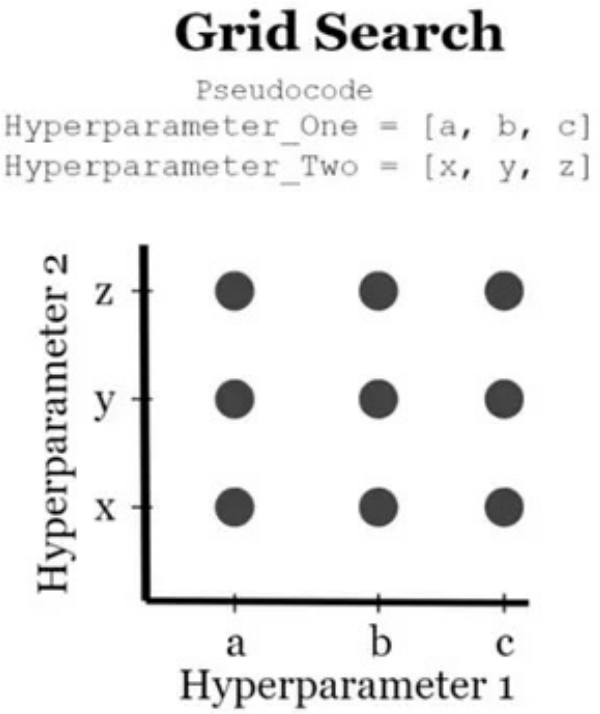
# Grid search

- Two main types of parameters:
  - Can be estimated from data:
    - Weights in logistic regression
    - Coefficients in ordinary linear regression
  - Hyperparameters / tuning parameters
    - Parameters needing optimisation
    - Regularization parameters
    - Learning rates
    - Algorithm type/version



# Grid search

- Brute-force exhaustive search through a grid of possible hyperparameter value combinations




- Important choices:
  - Parameters to be tuned
  - Range of parameters
  - Type of scoring
  - Type of validation
- Proper validation is needed to avoid overfitting
- Grid selection will affect speed! Alternative: Randomized search:
  - Stochastic search for optimal parameter values, can use probability distributions

# GridSearchCV – Simple Python Example

You want to **optimize a model** with two hyperparameters:

- `C` **(Regularization Parameter):** [0.1, 1, 10]
- `Gamma` **(Kernel Coefficient):** [0.01, 0.1, 1]

```python
from itertools import product

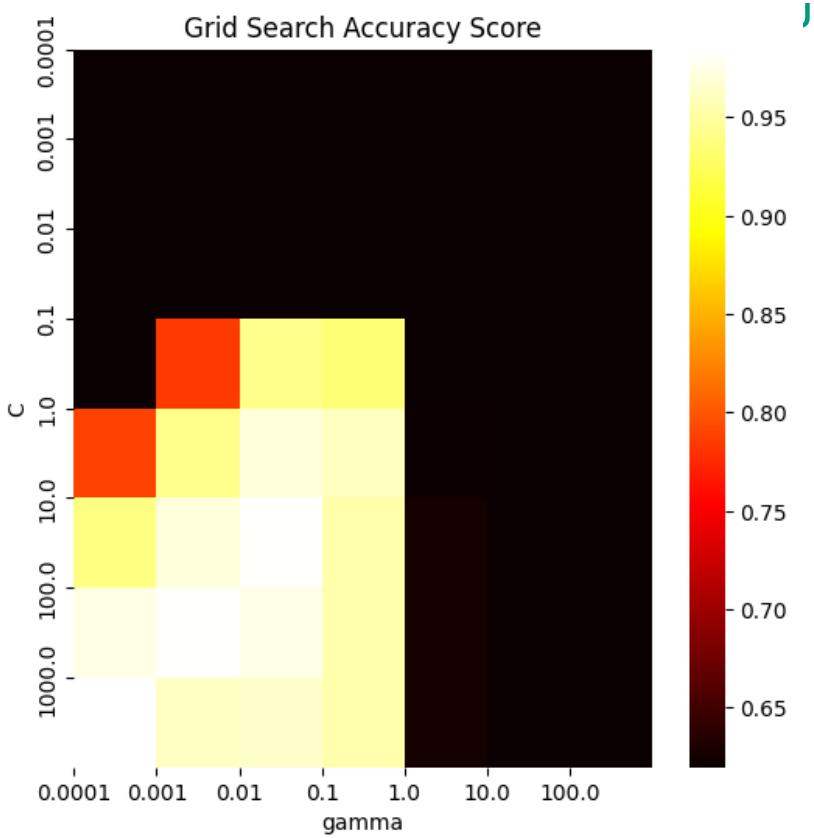
C_values = [0.1, 1, 10]
Gamma_values = [0.01, 0.1, 1]

combinations = list(product(C_values, Gamma_values))
print("Total combinations (Grid Search):", len(combinations))
print("Combinations:", combinations)
```

```
Total combinations (Grid Search): 9
Combinations: [(0.1, 0.01), (0.1, 0.1), (0.1, 1), (1, 0.01), (1, 0.1), (1, 1), (10, 0.01), (10, 0.1), (10, 1)]
```

# GridSearchCV

- Setup with:
  - Pipeline
  - Parameters
  - Scoring
  - Cross-validation




- Returns all model scores for the grid
- Returns a decision_function(X), containing the model trained with optimal parameters for direct application to new data

- The optimum might not be the most robust alternative!

# RandomizedSearchCV

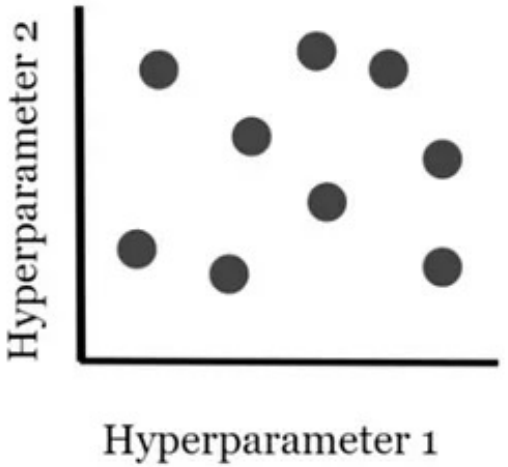
- We don’t specify a grid of hyperparameter value combinations to search through exhaustively
- We specify a set of possible values for each parameter, often a continuous range
- At each iteration the parameter values are sampled randomly
- Each range of possible parameter values also needs a corresponding **probability distribution**
  - If no distribution is specified, the samples are drawn from a uniform distribution
- We also specify a maximum set of iterations for the search
- Can anyone think of any advantages of random search over exhaustive grid search?

# RandomizedSearchCV

**Random Search**

Pseudocode

```
Hyperparameter_One = random.num(range)
Hyperparameter_Two = random.num(range)
```




- Setup with:
  - Pipeline
  - Parameters/**distributions**
  - Scoring
  - Cross-validation
- Random sampling of parameter values
- If at least one parameter is given as a distribution, sampling is done with replacement

# RandomizedSearchCV – Simple Python Example

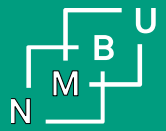
You want to **optimize a model** with two hyperparameters:

- `C` **(Regularization Parameter):** [0.1, 1, 10]
- `Gamma` **(Kernel Coefficient):** [0.01, 0.1, 1]

```
import random

combinations = list(product(C_values, Gamma_values))
random_combinations = random.sample(combinations, 4)
print("Total combinations (Random Search):", len(random_combinations))
print("Random Combinations:", random_combinations)
```

```
Total combinations (Random Search): 4
Random Combinations: [(1, 0.01), (1, 0.1), (10, 0.1), (1, 1)]
```


U
B
M
N